

KLUSTER

Technical Logic & Application Flow

Making Nigeria's Informal Economy Visible

Squadco Hackathon 2026

Backend: FastAPI + SQLAlchemy + PostgreSQL (Supabase)

Payments: Squad API AI: Google Gemini 2.5 Flash

1. What Is Kluster?

Kluster is a financial intelligence platform built for Nigeria's informal economy. The informal economy - market associations, cooperative societies, ajo (contribution) groups, and trade guilds - moves billions of naira every year, but because activity happens in cash and word-of-mouth, it is invisible to banks and investors. These businesses cannot access credit, and workers cannot demonstrate their economic value.

Kluster solves this by giving each cluster member a Squad virtual bank account, tracking every payment into that account, and turning that raw transaction data into intelligence: credit scores, demand signals, and economic profiles. It also connects growing clusters with job seekers who can help them scale.

Three types of users

- Cluster Leader - signs up, creates a cluster, adds members. Has access to cluster-level dashboards, health scores, and demand signals.
- Cluster Member - pre-registered by a leader, activates their own account later. Receives payments via Squad virtual account. Has their own activity score and can apply for micro-loans.
- Job Seeker - self-registers with their skills and location. Gets AI-matched to clusters that are growing and need help. Can apply to clusters where they see opportunities.

2. Tech Stack

Backend: FastAPI (Python) - async, auto-generated Swagger docs at /docs

Datab

All endpoints use plain Pydantic `response_model=` declarations. Every response is a typed schema instance - no generic wrappers. This means Swagger shows full response shapes for every endpoint.

3. Data Models

clusters

Represents a registered informal trade group.

id	UUID PK	Auto-generated
name	String	e.g. 'Alaba Phone Repair Association'
type	String	cooperative / market_association / ajo / guild
leader_member_id	UUID FK -> members	Points to the leader's Member row
location	String	e.g. 'Lagos'
description	String?	Optional description
languages	ARRAY(String)	e.g. ['Yoruba', 'English']

members

One row per person who belongs to a cluster. Leaders and regular members share this table - distinguished by role_in_cluster.

id	UUID PK	
auth_user_id	UUID?	Supabase auth user ID - null until member activates
first_name / last_name	String	
phone	String	Nigerian format 08011223344
cluster_id	UUID FK -> clusters	Which cluster this member belongs to
role_in_cluster	String	'leader' or 'member'
bank_account_number	String?	GTBank account for loan disbursement
squad_virtual_account_id	String?	Virtual account number for receiving payments
job_seeker_id	UUID FK -> job_seekers?	Links member to job seeker profile if they have one

job_seekers

Self-registered people looking for gig work within informal clusters.

id / auth_user_id	UUID	
first_name / last_name / phone	String	
bank_account_number	String?	GTBank account for receiving payments
skills	ARRAY(String)	e.g. ['phone repair', 'cashier', 'delivery']
language / location / bio	String	Used for matching
is_matched	Boolean	True once accepted by a cluster
matched_cluster_id	UUID FK -> clusters?	Set when a cluster accepts their application

transactions

Every payment into a member's Squad virtual account becomes a Transaction row, written by the webhook handler.

member_id	UUID FK -> members	Who received the payment
cluster_id	UUID FK -> clusters	Their cluster (for cluster-level aggregation)
amount	Float	In Naira
transaction_type	String	'credit' (only credits are currently recorded)
sender_ref	String?	Who paid - used for customer diversity scoring
squad_transaction_ref	String?	Squad's own reference ID
timestamp	DateTime	When the payment happened

demand_signals

Each time demand detection runs, it may write one or more signal rows. Signals are not deduplicated - each run creates fresh rows.

cluster_id	UUID FK -> clusters	
signal_type	String	rapid_growth / new_customer_influx / high_activity
strength	Numeric	0.0 to 1.0 - how strong the signal is
recommended_skills	ARRAY(Text)	AI-inferred or keyword-matched role labels
detected_at	DateTime	Server timestamp

scores

Every time a score is computed (cluster health or member activity), a new row is written. This builds a history of scores over time.

entity_type	String	'cluster' or 'member'
entity_id	UUID	The cluster_id or member_id being scored
score_decimal	Float	Final score 0-100
breakdown_json	JSONB	Per-metric breakdown - points earned vs max per category
calculated_at	DateTime	Note: typo in column name - do not change, DB has it as-is

financial_products

Records every loan disbursement. Created by POST /financial/disburse after Squad Transfer succeeds.

member_id	UUID FK -> members	
type	String	'loan' (only type currently)
amount	Numeric	In Naira
status	String	'pending' or 'success'
squad_transfer_ref	String?	Squad's transfer reference for audit

applications (PENDING - table not yet created)

New table to support the job seeker application flow. Run the SQL below in Supabase before building this feature.

```
CREATE TABLE IF NOT EXISTS applications (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  job_seeker_id UUID NOT NULL REFERENCES job_seekers(id) ON DELETE CASCADE,
  cluster_id UUID NOT NULL REFERENCES clusters(id) ON DELETE CASCADE,
```

```
status VARCHAR NOT NULL DEFAULT 'pending',  
created_at TIMESTAMP DEFAULT NOW()  
);
```

4. Authentication Flow

How JWT works in Kluster

Kluster uses Supabase Auth to manage user accounts. When a user signs up, their personal details (first_name, last_name, phone, bank_account_number) are stored inside the JWT token itself in a field called user_metadata. This means the backend can read the user's profile from their token without needing a separate database lookup.

Leader signup + cluster creation (2-step flow)

- Step 1: POST /auth/signup - leader provides email, password, name, phone. Backend calls Supabase and stores these in user_metadata.
- Step 2: POST /auth/login - leader gets a Bearer token.
- Step 3: POST /clusters/create - leader sends cluster details (name, type, location). Backend reads leader's name and phone FROM THE TOKEN, creates the cluster, creates a Member row for the leader with role='leader', and immediately calls Squad to create a virtual account for them.

The key insight: the leader never re-enters their name or phone. They're in the JWT.

Member activation (2-step flow)

- Step 1: Leader calls POST /clusters/members with the member's name, phone, and optionally bank_account_number. This creates a Member row in the DB with auth_user_id = NULL.
- Step 2: The member themselves comes to POST /auth/member-activate, provides email + password + phone. The backend finds their existing Member row by phone, creates their Supabase account, and links the new auth_user_id to the existing row.

This separation exists because the leader knows who they want in their cluster (name + phone), but the member controls their own login credentials. The phone number is the linking key.

Job seeker signup (1-step)

- POST /auth/job-seeker-signup - does everything in one call: creates Supabase account AND creates the JobSeeker DB row. Job seekers don't need to be pre-registered by anyone.

Token decoding

All protected endpoints use get_current_user (returns UUID) or get_current_user_profile (returns dict with UUID + name + phone). The JWT signature is NOT verified - this is intentional for the hackathon. In production, you would verify against the Supabase JWT secret.

5. Squad API Integration

Virtual Accounts

Every member - leader and regular member alike - gets a Squad virtual bank account when they are added to the system. The virtual account number is a GTBank account number that the member shares with their customers. When a customer pays into that number, Squad fires a webhook event to Kluster.

- Virtual account creation: called immediately after creating a Member row
- The returned virtual_account_number is stored on the Member as squad_virtual_account_id
- If SQUAD_API_KEY is not set, the system returns a mock account number (for local dev)

Webhook Flow (how transactions get recorded)

This is the heart of the data pipeline:

1. Customer pays into member's virtual account number at any GTBank branch or via transfer
2. Squad detects the payment and sends a POST request to POST /webhooks/squad
3. Kluster checks transaction_indicator == 'C' (credit only - ignores debits)
4. Kluster finds the Member by matching virtual_account_number to squad_virtual_account_id
5. Kluster writes a Transaction row with amount, sender_ref (customer name), timestamp
6. This transaction data feeds the scoring engine and demand detection

The sender_overrides Trick (important for demo)

Squad's sandbox simulate endpoint does not allow you to specify a custom sender name - it always generates the same name. This breaks customer diversity scoring (which counts unique sender names). To work around this:

- POST /sim-transfer accepts an optional sender_name field
- If provided, it is stored in a module-level dict: sender_overrides[virtual_account_number] = sender_name
- Squad then fires a webhook (asynchronously)
- The webhook handler does: sender_ref = sender_overrides.pop(virtual_account_number, None) or payload.get('sender_name')
- The override is consumed exactly once - subsequent webhooks to the same account use Squad's default name

This is an in-memory bridge. If the server restarts between sim-transfer and webhook, the override is lost.

Loan Disbursement (Squad Transfer API)

When a member is approved for a loan (POST /financial/disburse), the backend calls Squad's Transfer API to send money to their bank account. Key details:

- Bank code is hardcoded as '058' (GTBank) - all accounts are assumed to be GTBank since Squad is built on GTBank infrastructure
- Amount is sent in kobo (Naira x 100) to Squad
- A FinancialProduct row is created to record the disbursement and Squad's transfer reference
- Falls back to a mock response if SQUAD_API_KEY is not set

6. Scoring Engine

The scoring engine is one of the core value propositions of Kluster - it transforms raw transaction data into a credit-worthiness signal. There are two scores: one for the cluster as a whole, and one for each individual member.

Cluster Health Score (0-100)

Answers: 'Is this cluster economically healthy and growing?' Used as one half of loan eligibility.

Metric	Max Points	How It's Calculated
Active Member Rate	25	% of members who had at least one credit transaction in the last 30 days
Volume Trend	25	% growth in total credit volume vs the prior 30-day period
Transaction Consistency	20	Coefficient of variation across 8 weekly buckets. Lower variance = more consistent =
Customer Diversity	15	Unique sender names in last 30 days, capped at 10 senders for full points
Member Retention	15	% of previously active members who are still active this period

Member Activity Score (0-100)

Answers: 'Is this specific member economically active relative to their peers?' The other half of loan eligibility.

Metric	Max Points	How It's Calculated
Transaction Volume	30	Member's volume vs cluster average. Getting 2x the average = full points
Consistency	25	Same CV calculation as cluster - lower variance in weekly income = higher score
Customer Count	25	Unique payers in last 30 days, capped at 5 for full points
Growth Trend	20	Member's own volume growth vs their prior 30-day period

Loan Eligibility (Dual-Score Model)

Both scores must independently reach 40/100. A strong cluster cannot compensate for a weak member, and a high-performing member cannot be dragged down by a failing cluster. This design prevents free-riding in both directions.

Eligible = (individual_score >= 40) AND (cluster_score >= 40)

Max Loan = NGN 50,000 x (individual_score / 100) x (cluster_score / 100)

Example: individual=70, cluster=80 -> 50,000 x 0.70 x 0.80 = NGN 28,000

If a member is not eligible, improvement_tips identify the specific weakest metric for both scores - rejection becomes a roadmap, not a dead end.

7. Demand Signal Detection

Demand signals answer: 'Is this cluster growing fast enough that it needs to hire more people?' Rather than requiring employers to post job listings, Kluster infers labor demand directly from financial activity. Three types of signals are detected:

Signal 1: rapid_growth

- Condition: credit volume in the last 14 days grew by 40% or more vs the prior 14 days
- Strength: $\min(\text{growth_rate} / 2, 1.0)$ - a 100% growth = strength 0.5, 200% = strength 1.0
- Requires: previous period had some volume ($\text{prev_volume} > 0$)
- Interpretation: The cluster's revenue is accelerating. They need operational support to handle the growth.

Signal 2: new_customer_influx

- Condition: unique sender names grew by 30%+ vs prior 14 days
- Fallback: if no previous senders exist AND total recent transactions ≥ 3 , signal fires at strength 0.6
- Note: The fallback threshold was set to 3 transactions (not unique senders) because Squad sandbox always generates the same sender name, making unique sender count unreliable for demo purposes
- Interpretation: New customers are arriving. The cluster needs front-facing help - sales, cashiers, reception.

Signal 3: high_activity

- Condition: 50 or more credit transactions in the last 30 days
- Strength: $\min(\text{tx_count} / 200, 1.0)$ - 200+ transactions = full strength
- Interpretation: The cluster is simply very busy, regardless of growth. They need operational staff.

Skill Inference (Gemini AI)

For each detected signal, Kluster asks Gemini to infer 4 types of workers the cluster likely needs, based on the cluster's name, type, signal type, and transaction data. Examples:

- 'Alaba Phone Repair Association' + rapid_growth -> ['repair assistant', 'parts sourcer', 'dispatch rider', 'cashier']
- 'Balogun Fashion Cooperative' + new_customer_influx -> ['seamstress', 'fabric sourcer', 'sales assistant', 'delivery worker']

If Gemini is unavailable (no API key), the system falls back to a keyword map that checks the cluster name for industry keywords (repair, tailor, food, farm, beauty, etc.) and returns a preset skill list. If no keyword matches, it falls back to the cluster type (market_association, cooperative, ajo, guild) to pick a generic skill set.

8. AI-Powered Job Seeker Matching

How Matching Works (GET /matching/opportunities)

This endpoint is the bridge between labor supply (job seekers) and demand (clusters with active signals). It runs every time a job seeker requests their opportunities list:

- 1. Identify the logged-in job seeker from their JWT token
- 2. Query all clusters in the database
- 3. For each cluster, run demand signal detection to find active signals
- 4. For each signal, check if the job seeker's skills overlap with the signal's recommended skills (loose substring match - 'repair' matches 'repair assistant')
- 5. Also check if the job seeker's location matches the cluster's location (substring match in either direction)
- 6. If either condition is true, it's a match
- 7. For each match, call Gemini to generate a one-sentence explanation of why this is a good fit
- 8. Sort all matches by signal strength (strongest first) and return

Example result: 'Adaeze's phone repair skills directly address Alaba Electronics Association's rapid 60% revenue growth, making her well-suited to support their expanding repair operations.'

AI Economic Profile (GET /members/me/profile)

For cluster members, Kluster generates a human-readable economic narrative using Gemini. The AI receives the member's score, transaction volume, unique customer count, and growth rate, then writes a 2-3 sentence professional summary that a bank or investor could use to evaluate them.

Example: 'Emeka Okafor is an active member of Nnewi Auto Parts Cooperative with an activity score of 74/100. Over the last 30 days, he recorded NGN 387,000 in revenue from 12 unique customers, reflecting a +23.4% growth trend driven by seasonal demand for spare parts.'

Falls back to a template-based profile if GEMINI_API_KEY is not configured.

9. Application Flow (PENDING - not yet built)

The application feature completes the end-to-end loop: job seeker discovers an opportunity -> applies -> cluster member reviews -> accepts. This is not yet implemented.

Planned Flow

- 1. Job seeker calls GET /matching/opportunities to see which clusters need their skills
- 2. Job seeker picks a cluster and calls POST /applications with {cluster_id}
- 3. All members of that cluster can call GET /applications/cluster to see who applied - response includes applicant's name, skills, and location
- 4. Any cluster member (not just the leader) can call POST /applications/{id}/accept
- 5. On acceptance: application status -> 'accepted', job_seeker.is_matched -> True, job_seeker.matched_cluster_id -> that cluster
- 6. Job seeker can see their application status via GET /applications/me

There is no reject endpoint - cluster members simply leave unwanted applications as 'pending'. There is no payment endpoint - the complexity of tracking service completion and member-initiated payment was out of scope for the hackathon timeline.

10. Full Endpoint Reference

Auth (/auth)

Endpoint	Auth	Description
POST /auth/signup	Cluster leader	Create Supabase account, store profile in JWT metadata
POST /auth/login	Anyone	Email + password -> JWT Bearer token
POST /auth/member-activate	Pre-registered member	Create Supabase account, link to existing Member row by phone
POST /auth/job-seeker-signup	Job seeker	Create Supabase account + JobSeeker DB row in one step

Clusters (/clusters)

Endpoint	Auth	Description
POST /clusters/create	Leader JWT	Create cluster + leader Member + Squad virtual account
GET /clusters/me	Leader JWT	Return leader's cluster
GET /clusters/all	Public	Return all clusters
POST /clusters/members	Leader JWT	Add member to cluster + create their Squad virtual account
GET /clusters/members	Leader JWT	List all members of the cluster
GET /clusters/health	Leader JWT	Run cluster health score, return 0-100 + breakdown
GET /clusters/demand	Leader JWT	Run demand signal detection, return signals + recommended skills
GET /clusters/transactions	Leader JWT	Return cluster transaction feed

Members (/members)

Endpoint	Auth	Description
GET /members/lookup	Public	Look up member by phone - returns name, cluster, virtual account
GET /members/me	Member JWT	Own profile
GET /members/me/transactions	Member JWT	Own transaction history
GET /members/me/score	Member JWT	Own activity score + breakdown
GET /members/me/profile	Member JWT	AI-generated economic profile narrative

Job Seekers (/job-seekers)

Endpoint	Auth	Description
GET /job-seekers/me	Job seeker JWT	Own job seeker profile

Matching (/matching)

Endpoint	Auth	Description
GET /matching/opportunities	Job seeker JWT	AI-matched cluster opportunities based on skills + location

Financial (/financial)

Endpoint	Auth	Description
POST /financial/prequalify	Member JWT	Dual-score loan eligibility check + max loan amount
POST /financial/disburse	Member JWT	Disburse loan via Squad Transfer -> member's bank account

Webhooks (/webhooks)

Endpoint	Auth	Description
POST /webhooks/squad	Squad (no auth)	Receive payment events, log Transaction rows

Sim Transfer (/sim-transfer)

Endpoint	Auth	Description
POST /sim-transfer/	No auth	Trigger Squad sandbox simulate payment (demo/testing only)

Applications (/applications) - PENDING

Endpoint	Auth	Description
POST /applications	Job seeker JWT	Apply to a cluster (body: {cluster_id})
GET /applications/me	Job seeker JWT	See own applications and statuses
GET /applications/cluster	Any member JWT	See all applications for own cluster
POST /applications/{id}/accept	Any member JWT	Accept an applicant

11. Environment Variables

Variable	Status	Description
DATABASE_URL	Required	PostgreSQL connection string from Supabase
SUPABASE_URL	Required	e.g. https://xyz.supabase.co
SUPABASE_ANON_KEY	Required	Supabase public anon key
SUPABASE_JWT_SECRET	Optional	Not currently used (signature verification disabled)
SQUAD_API_KEY	Required for payments	Squad sandbox API key
SQUAD_BENEFICIARY_ACCOUNT	Required for virtual account	Your Squad settlement account number
GEMINI_API_KEY	Optional	Google Gemini key - falls back to templates if not set
ALLOWED_ORIGINS	Optional	Comma-separated CORS origins - defaults to * for dev

12. Pending Work (Next Session)

A. bank_account_number field (partially done)

The MemberBase and MemberView schemas already have bank_account_number. Still needed:

- Add column to Member model (models/member.py)
- Run: ALTER TABLE members ADD COLUMN IF NOT EXISTS bank_account_number VARCHAR;
- Add to SignupRequest in auth.py, store in Supabase user_metadata
- Extract in get_current_user_profile, save to leader Member in create_cluster
- Add to JobSeeker model, JobSeekerSignupRequest, JobSeekerView, job_seeker_signup
- Run: ALTER TABLE job_seekers ADD COLUMN IF NOT EXISTS bank_account_number VARCHAR;
- Make account_number optional in DisburseRequest (fall back to member.bank_account_number)
- Remove bank_code from DisburseRequest entirely - hardcode '058' in financial.py

B. Application feature (not yet started)

- Create models/application.py
- Create schemas/application.py (ApplicationCreate, ApplicationView)
- Create routers/applications.py with 4 endpoints
- Update models/__init__.py, routers/__init__.py, main.py
- Run the applications table SQL migration in Supabase

C. Deploy + Seed

- Deploy backend to Render
- Register deployed webhook URL in Squad dashboard
- Seed 30-60 days of spread transactions before presentation to get meaningful scores